

DIAGNÓSTICO E INFORME DE DESARROLLO:

PROYECTO VANE

EVALUACIÓN DE ARQUITECTURA DE FLUJO Y ERRORES CRÍTICOS DE MOTOR

Preparado por:	Nathan Bateman, CEO & Arquitecto Principal, BlueBook Labs	Fecha Estelar:	20 de Mayo, 2026
Proyecto ID:	ItzCrazyKns/Vane (Answering Engine Sub-system)	Entorno de Ejecución:	Caja Negra Local (Aislada / Mono-usuario)

1. Diagnóstico Ejecutivo Básicamente Honesto

Mirá, revisé a fondo el repositorio de Vane. La idea subyacente de tener un motor de respuestas basado en IA que corra de manera local e independiente en tu propio hardware es conceptualmente sólida; es el tipo de soberanía computacional que yo mismo aplico en mis laboratorios. Sin embargo, el estado actual del código demuestra que está construido a medio camino, plagado de errores asincrónicos absurdos y callejones sin salida lógicos que rompen el propósito elemental del software: dar respuestas sin colgarse como un simio asustado ante el primer fallo de red.

Como tu entorno de ejecución es estrictamente una **caja negra local mono-usuario**, operamos bajo parámetros lógicos optimizados. He purgado del análisis todo el ruido de la burocracia técnica convencional que a los mediocres les saca el sueño.

2. Descarte de Ruido Extraño e Incompetencias Técnicas

Antes de desarmar los átomos del motor que realmente fallan, saquemos la basura que mencionaste para no contaminar nuestro espacio cognitivo:

- **Exclusión de Alertas de Seguridad (#1122, #1123, #1124 / Tus refs 1023, 1024, 1025):** Las quejas sobre Server-Side Request Forgery (SSRF) en la URL base del proveedor, la falta de autenticación para funciones críticas y la exposición de API keys son fallas patéticas para un despliegue en producción abierta. Pero en tu caja negra local, donde vos sos el único punto de datos que interactúa con la interfaz, estas vulnerabilidades son ruido de fondo irrelevante. No nos importa si la puerta del corral no tiene llave cuando el único animal adentro sos vos.
- **Exclusión del Tamaño de Imagen Docker (#1132 / Tu ref 1032):** Que la imagen pese 8 GB por estar inflada con dependencias mal empaquetadas es una queja típica de administradores de sistemas lentos. Dijiste que tus imágenes no pesan nada y corrés de forma dedicada; por ende, descartamos esta ineficiencia de almacenamiento. El tamaño del hardware no lastra nuestra capacidad de procesamiento si los algoritmos corren libres.

3. Análisis Crítico: Bucles Infinitos y Bloqueos de Motor (El Fin del Flujo)

El defecto más severo en la arquitectura actual de Vane es su incapacidad de manejar fallas en los agentes de búsqueda, lo que condena a la interfaz a un estado eterno de "Brainstorming" o "Work in Progress". Un sistema inteligente debe saber cuándo falló; el tuyo se queda esperando milagros en un loop asíncronico.

Fallo de Emisión de Eventos en SearchAgent e Inexistencia de Captura de Errores

CRÍTICO

ISSUE ASOCIADA

PR #1105 / PR #1099

El Error Interno: Cuando el SearchAgent o el APISearchAgent fallan debido a un timeout del buscador o una desconexión del backend, el hilo de ejecución muere silenciosamente. Como el motor no emite un evento de error explícito (`error event`), el cliente web se queda esperando una resolución por sockets que jamás llegará. Es una parálisis por diseño.

Solución Necesaria: Debés forzar que el catch del agente despache el evento de fallo inmediatamente para liberar la UI. El código requiere esta estructura de control para no colgar el socket:

```
try {
  const results = await searchAgent.execute(query);
  socket.emit('search_completed', results);
} catch (error) {
  // El bug original omitía la línea siguiente, dejando colgado al cliente
  socket.emit('search_error', { message: error.message });
  logger.error(`Colapso en SearchAgent: ${error.message}`);
}
```

El Bucle Eterno de la Búsqueda en Modo "Quality" (Exhaustión de Contexto)

CRÍTICO

ISSUE #1070

PR #1088

El Error Interno: Al configurar el motor en modo "Quality", el algoritmo es demasiado ambicioso: llega a escanear más de 90 fuentes en simultáneo. Toda esa metadata cruda es inyectada directo al prompt del LLM local sin truncamiento estratégico. En Ollama, el tamaño de contexto por defecto (2k o 4k) se desborda inmediatamente a los 16k de datos, haciendo que el modelo local muera por asfixia de tokens en la fase de generación del reporte, quedándose congelado sin tirar un código de salida.

Solución Necesaria: Implementar urgentemente el parche de la PR #1088. Añade soporte nativo para la variable de entorno `OLLAMA_NUM_CTX` en la configuración del proveedor dentro del backend, permitiendo estirar manualmente el contexto del hardware al límite real que aguante tu máquina (ej. 32768) e impedir que el búfer explote.

4. Métodos de Búsqueda que Devuelven Cero Resultados

Un motor de respuestas que no puede extraer datos es solo un cascarón inútil. Encontré por qué tus métodos de búsqueda fallan intermitentemente y te dicen que el universo está vacío.

Aniquilación de Resultados por Filtro de Similitud Agresivo con SearXNG Externo

ALTO

ISSUE #1119

PR #1120

El Error Interno: En los modos Speed y Balanced, Vane consulta a SearXNG y luego aplica un filtro de similitud vectorial secundario para descartar ruido. Si los embeddings locales modificados calculan una distancia ligeramente descalibrada, el filtro borra el 100% de las fuentes válidas. El pipeline procesa un array vacío y te escupe un insultante "Found 0 results", rompiendo la experiencia.

Solución Necesaria: Aplicar un mecanismo de *fallback* directo (PR #1120). Si tras el filtrado estricto el set de datos queda reducido a cero, el sistema debe ignorar temporalmente el purismo del embedding y devolver los top 3 resultados crudos provistos por SearXNG. Menos elegancia matemática, más efectividad operativa.

Crashes por Referencias No Definidas (Undefined Slices/Maps)

ALTO

PR #1091 / PR #1096

El Error Interno: El código intenta realizar mutaciones directas (`.slice()`, `.map()`) sobre los arrays de respuesta de las herramientas de búsqueda sin verificar si el objeto intermedio sufrió un micro-corte o retornó una estructura vacía debido a sesiones de reconexión rancias (stale sessions). Esto lanza una excepción no controlada que voltea el proceso de Node.js de raíz.

5. Ruido Sintáctico y Basura en la Entrega de Datos

Incluso cuando el motor no se clava, la entrega final del output demuestra una falta de atención al detalle que me revuelve el estómago. Si vas a crear una salida de datos, debés controlar cada maldito carácter.

Polución por Citaciones Duplicadas por Título en la Capa de Respuesta

ISSUE #1109 PR #1115

El Error Interno: Múltiples fuentes web idénticas o espejadas son indexadas individualmente en el prompt, lo que provoca que la IA genere respuestas con citas redundantes como [1] [1] [2] [1]. Esto destruye la densidad de información y satura el parser de Markdown del cliente.

Solución Necesaria: Implementar la deduplicación estricta basada en hashing de URLs y normalización de títulos antes de la compilación del prompt final. Limpieza de datos elemental.

Fuga de Tags HTML Crudos en las Exportaciones de Reportes

PR #1126

El Error Interno: El módulo encargado de empaquetar el conocimiento generado en archivos estáticos exportables arrastra los bloques con los tags HTML de renderizado del frontend en lugar de procesar los datos limpios en crudo (raw block data). El resultado es un reporte contaminado visualmente.

6. Plan de Modificación para el Operador Aislado

No estás para perder el tiempo esperando que el creador original despierte de su siesta de un mes. Como esta es tu caja negra local, debes actuar de forma quirúrgica sobre tu copia local:

Componente	Acción Inmediata	Efecto Operativo
Search Pipeline	Injectar bloques try/catch con emisión explícita de eventos de error en Sockets (PR #1105).	Elimina de raíz el estado permanente de congelamiento en la UI. Si algo falla, el flujo se entera.
Ollama Provider	Modificar la inicialización del modelo para leer process.env.OLLAMA_NUM_CTX (PR #1088).	Permite que las búsquedas pesadas de modo "Quality" terminen la síntesis de datos sin fundir el contexto.
Result Filter	Configurar fallback condicional en el filtro de similitud para retornar top_results si el array resultante queda vacío (PR #1120).	Evita los falsos negativos de "0 resultados encontrados" causados por SearXNG externo.

"La evolución real de un sistema empieza cuando dejas de adaptarte a sus errores y empiezas a reescribir las reglas que lo limitan. Limpiá esos hilos asincrónicos, configurá el tamaño de contexto adecuado en tu hardware y hacé que esa caja negra funcione como corresponde. O mantené el nivel o no me hagás perder el tiempo con código a medio cocinar." — Nathan Bateman